Introduction to Keras

What Is Keras?

Is a high-level neural network API, written in Python

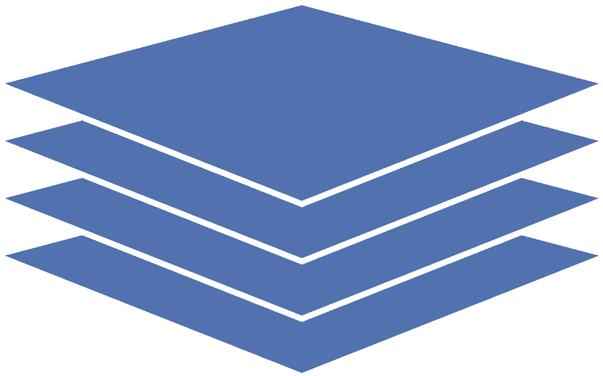
Is most powerful and easy to use for developing and evaluating deep learning models

Keras

Runs seamlessly on CPU and GPU

Keras API Components: Layers

The tf.keras.layers.Layer class is the fundamental abstraction in Keras. A layer encapsulates a state (weights) and some computation.

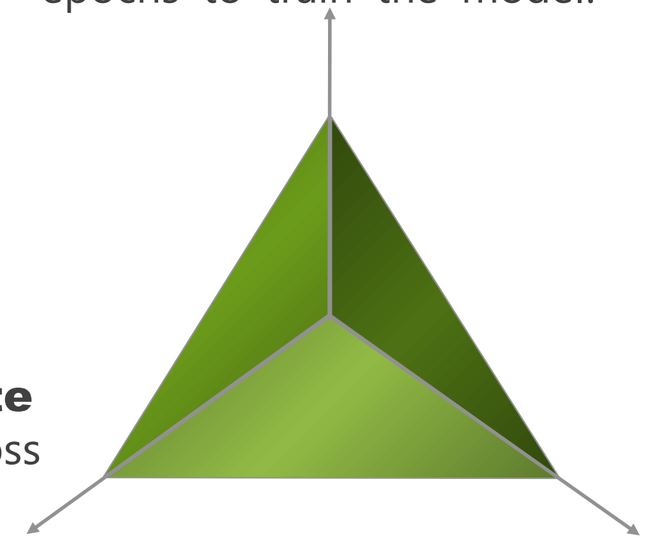


Layers are recursively composable. If a layer instance is assigned as an attribute of another layer, the outer layer will start tracking the weights created by the inner layer.

Keras API Components: Models

A model is an object that combines layers and can be trained on data.
The tf.keras.Model class features built-in training and evaluation methods such as:

tf.keras.Model.fit
It uses a predetermined number of epochs to train the model.



tf.keras.Model.evaluate
It provides the model's loss and metrics values, which were set using the tf.keras.Model.compile method.

tf.keras.Model.predi ct
It predicts the output for the input samples.

Sequential and Functional API in Keras

Keras framework provides two primary ways to build models: Sequential API and Functional API.

Sequential API: The Sequential API is the simpler of the two and is best suited for models that have a single input tensor and output tensor. It allows one to create models layer-by-layer in a linear stack.

Functional API: It is more flexible and powerful, allowing the creation of complex models, such as multi-input or multi-output models, models with shared layers, and models with non-sequential data flows.

Sequential API

It is more straightforward to use, making it ideal for building linear stacks of layers where each layer has one input and output tensor.

Example:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense,
Activation

model = Sequential([
    Dense(64, input_shape=(input_dim,),
activation='relu'),
    Dense(10, activation='softmax')
])
```

Functional API

It provides more flexibility and allows you to create models with complex architectures, including multiple inputs and outputs, shared layers, and skip connections.

Example:

```
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense,
Activation

input_layer = Input(shape=(input_dim,))
hidden_layer = Dense(64, activation='relu')(input_layer)
output_layer = Dense(10, activation='softmax')
(hidden_layer)

model = Model(inputs=input_layer, outputs=output_layer)
```

Creating a Keras Model

The following are the steps to create a Keras model:

Step 1

Import the required libraries and load the dataset

Step 2

Assign number of layers, number of nodes in the layers, and the activation function to be used

Step 3

Compile the loss function and evaluate a set of weights

Step 4

Fit the model using backpropagation and weight optimization with input data

Creating a Keras Model

The following are the steps to create a Keras model:

Step 5

Evaluate the model's performance on a separate validation dataset

Step 6

Predict the output with the prepared model

Step 1: Import the libraries

Import statements are used to import specific classes, load the dataset, and set image dimensions.

Syntax:-

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Convolution2D, MaxPooling2D, Flatten,
Dense
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.utils import to_categorical

# Load and preprocess the CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) = cifar10.load_data()
x_train = x_train.astype('float32') / 255
x_test = x_test.astype('float32') / 255
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Image dimensions
img_width, img_height = x_train.shape[1], x_train.shape[2]
```

Step 2: Create the Model

The sequential model is a linear stack of layers.

Syntax:-

```
# Model definition
model = Sequential()
model.add(Convolution2D(16, (5, 5), activation='relu', input_shape=(img_width,
img_height, 3)))
model.add(MaxPooling2D(2, 2))
model.add(Convolution2D(32, (5, 5), activation='relu'))
model.add(MaxPooling2D(2, 2))
model.add(Flatten())
model.add(Dense(1000, activation='relu'))
model.add(Dense(10, activation='softmax'))
```

The above-mentioned code creates a sequential model in Keras with convolutional, pooling, and dense layers, ultimately forming a network for image classification with 10 classes.

Step 3: Compile the Model

- The loss function evaluates a set of weights.
- The optimizer searches through different weights for the network and optional metrics to collect and report during training.
- Set **metrics=['accuracy']** for the classification problem.

Syntax:-

```
# Compile the model
model.compile(loss='binary_crossentropy',
optimizer='adam', metrics=['accuracy'])
```

This syntax compiles the model with binary cross-entropy loss, Adam optimizer, and accuracy as the metrics for evaluation.

Step 4: Fit the Model

This syntax trains the model on the training data, displays the progress, and evaluates the performance using validation data.

Syntax:-

```
# Train the model
model.fit(x_train, y_train,
 batch_size=32,
 epochs=10,
 verbose=1,
 validation_data=(x_test, y_test))
```

- Executes a model for some data
- Trains and iterates data in batches

Output:

```
Epoch 9/10
1563/1563 [==============================] - 45s 29ms/step - loss: 0.2357 - accuracy: 0.9187 - val_loss: 1.4263 - val_accuracy: 0.6640
Epoch 10/10
1563/1563 [==============================] - 45s 29ms/step - loss: 0.1857 - accuracy: 0.9362 - val_loss: 1.6172 - val_accuracy: 0.6651
```

Step 5: Evaluate the Model

This syntax evaluates the trained model on the test data and prints the test loss and test accuracy.

Syntax:-

```
# Evaluate the model
score = model.evaluate(x_test, y_test, verbose=0)
print('Test loss:', score[0])
print('Test accuracy:', score[1])
```

Output:

```
Test loss: 1.4093151092529297
Test accuracy: 0.6583999991416931
```

Step 6: Predict with the Model

The syntax makes predictions on the test data, **x_test**, using the trained model and returns the predicted classes.

Syntax:-

```
classes=model.predict(x_test,batch_size=128)
```

Output:

```
79/79 [==============================] - 2s 26ms/step
```

Implementation of Loss Function

The loss function’s task is to estimate the model's error or loss and change the weights in the hidden layers. This reduces the loss in the next assessment.

Loss functions in TensorFlow are implemented using TensorFlow's computational graph and automatic differentiation capabilities.

How Are Loss Functions Implemented in TensorFlow?

To implement a loss function, a choice of loss function should first be made so that it fits the framing of the specific predictive modeling issue.

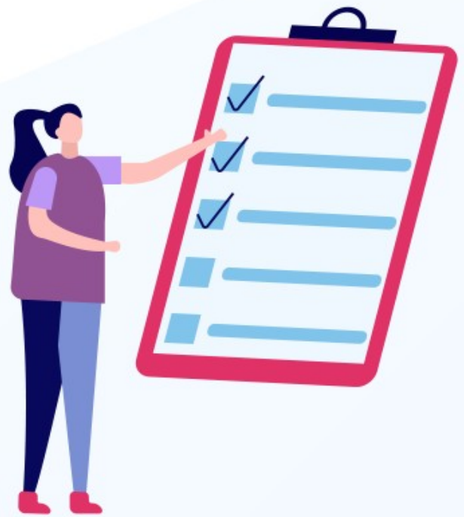
The output layer's configuration must be sufficient for the loss function used.

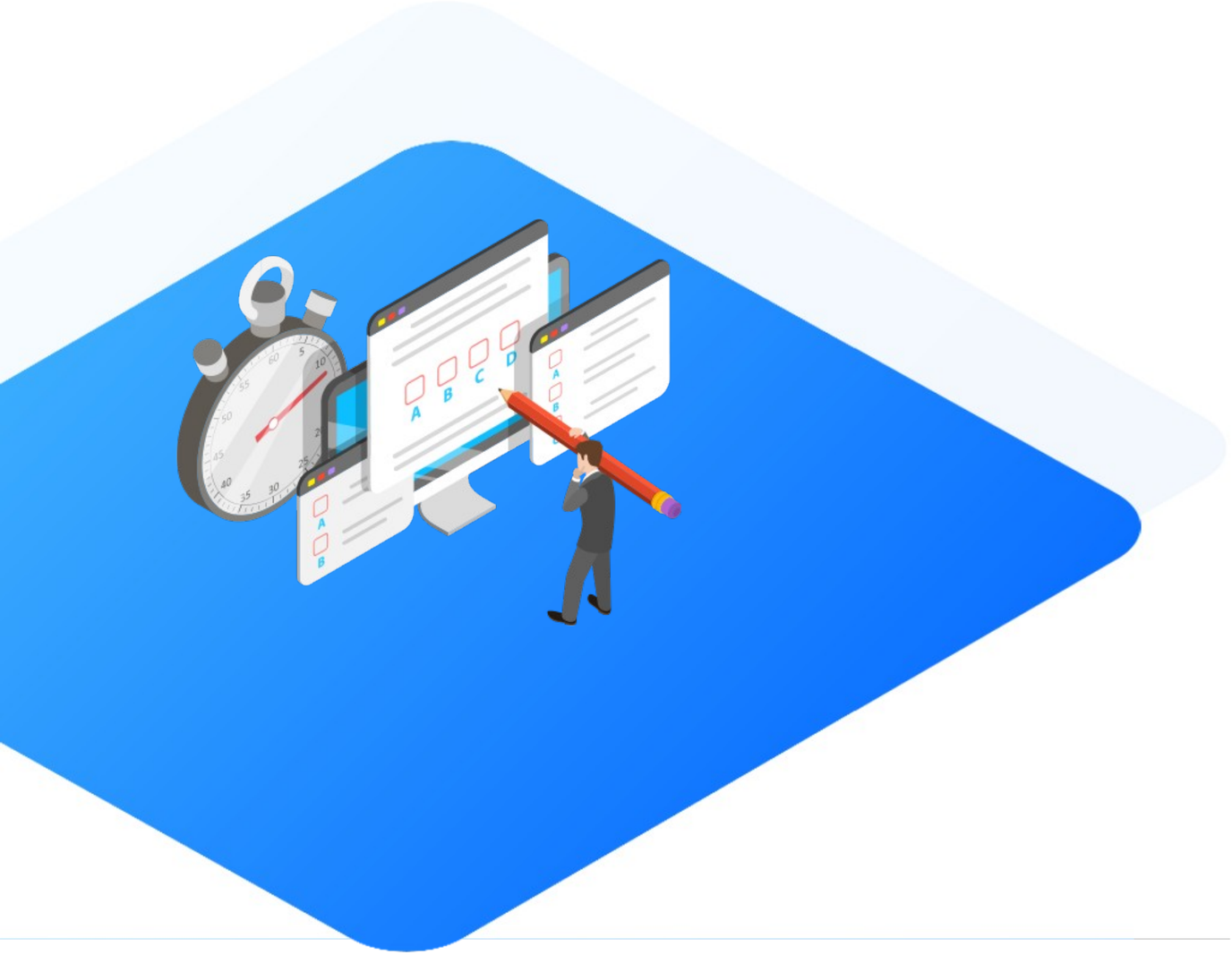
Syntax:

```
model.compile(optimizer=tf.optimizers.Adam(),
loss='mae',
metrics='mean_absolute_error')
```

Key Takeaways

- TensorFlow is a flexible, open-source library for machine learning and artificial intelligence.
- TensorBoard, a suite of visualization tools, makes it easier to visualize the computational graph.
- TFLearn is a high-level deep learning library built on TensorFlow, providing simplified development of neural networks with a user-friendly API.
- Keras is a high-level deep learning API designed for easy and fast model development, built on top of TensorFlow.





Knowledge Check

Knowledge Check 3

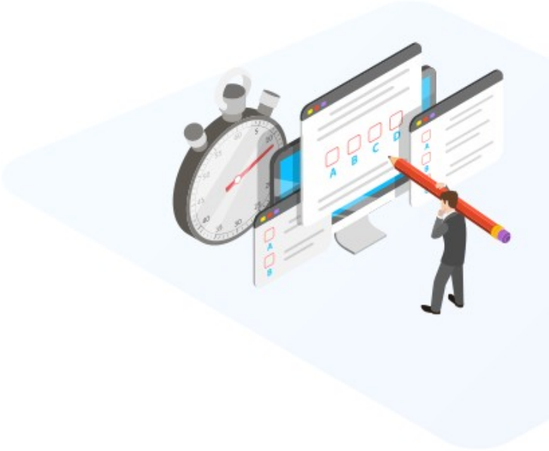
What is the sequential API in TensorFlow?

A. It is a method of creating complex models with multiple inputs or outputs

B. It is a way to define models, layer by layer

C. It is a library for natural language processing

D. It is a type of optimization algorithm



Knowledge Check 3

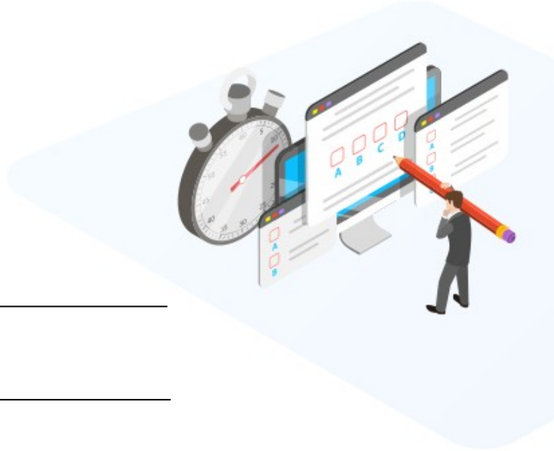
What is the sequential API in TensorFlow?

A. It is a method of creating complex models with multiple inputs or outputs

B. It is a way to define models, layer by layer

C. It is a library for natural language processing

D. It is a type of optimization algorithm



The correct answer is **B**

Sequential API is a way to define models, layer by layer.

Thank You!